# Observabilidad de Base de Datos con Stack de Monitoreo Real

**Arody Rodrigo Angel Sontay, Ian Gabriel Barrera Medina, Marlon Efraín García Raxón, Josué Samuel Patzán García**

*Universidad (Mariano Galvez de Guatemala)*
*INGENIERÍA EN SISTEMAS DE INFORMACIÓN Y CIENCIAS DE LA COMPUTACIÓN*
*Curso: Bases de Datos II*
*Docente: Wilmer Tezén*
*Semestre: 1/2026*

*Informe Académico*
*Laboratorio de Análisis de Rendimiento de Bases de Datos*

**Abstract:** En este trabajo se implementó un entorno de observabilidad para analizar el rendimiento de una base de datos PostgreSQL bajo carga. Se construyó un sistema basado en contenedores utilizando Docker Compose, integrando una API en Node.js, PostgreSQL, un exportador de métricas, Prometheus y Grafana. Para simular carga real, se utilizó Artillery, generando múltiples solicitudes concurrentes hacia la API. Los resultados permitieron observar métricas clave como conexiones activas, uso de índices, inserciones por segundo y acumulación de dead tuples. Se identificó cómo el uso adecuado de índices impacta directamente en el rendimiento de las consultas. Finalmente, se concluye que el monitoreo continuo mediante herramientas de observabilidad permite detectar ineficiencias que no son evidentes con análisis estáticos como EXPLAIN ANALYZE, facilitando la toma de decisiones para optimización.

## 1. Introducción

El rendimiento de una base de datos es un factor crítico en el desarrollo de aplicaciones modernas, especialmente en sistemas que manejan múltiples solicitudes concurrentes. Tradicionalmente, el análisis del rendimiento se realiza mediante herramientas como EXPLAIN ANALYZE, que permiten inspeccionar el plan de ejecución de consultas específicas. Sin embargo, este enfoque presenta limitaciones importantes, ya que no refleja el comportamiento del sistema bajo condiciones reales de carga ni permite observar métricas en tiempo real.

En entornos de producción, es necesario complementar este análisis con herramientas de monitoreo continuo que permitan visualizar el estado del sistema de manera dinámica. En este contexto, el uso de soluciones de observabilidad como Prometheus y Grafana permite recolectar y visualizar métricas relevantes del sistema.

Este trabajo tiene como objetivo analizar el rendimiento de PostgreSQL desde la perspectiva de la aplicación, utilizando pruebas de carga con Artillery y herramientas de monitoreo para identificar cuellos de botella y evaluar el impacto de los índices en el rendimiento.

## 2. Descripción del entorno

Para la realizacion de los experimentos se construyo un entorno completo de observabilidad de base de datos utilizando contenedores Docker orquestados mediante Docker Compose. Este entorno permite generar carga controlada sobre una API, monitorear el comportamiento interno de PostgreSQL y visualizar metricas del sistema en tiempo real.

El objetivo de esta arquitectura es simular un escenario real de producción en el cual una aplicación recibe trafico HTTP, ejecuta consultas sobre una base de datos y expone métricas

operativas que permiten analizar el rendimiento del sistema bajo carga.

El stack utilizado se compone de seis componentes principales: PostgreSQL, la API REST, `postgres-exporter`, Prometheus, Grafana y Artillery. Cada uno cumple un rol especifico dentro del flujo de observabilidad.

### *2.1. PostgreSQL*

PostgreSQL es el sistema gestor de base de datos relacional utilizado para almacenar y procesar los datos de la aplicacion. En este entorno se utilizo PostgreSQL version 16 ejecutandose dentro de un contenedor Docker.

La base de datos contiene dos tablas principales: `pedidos` y `clientes`. La tabla `pedidos` almacena la informacion de cada transaccion realizada en el sistema, incluyendo identificadores de cliente y producto, region, estado del pedido, cantidades, precios y marcas de tiempo. Esta tabla es el foco principal de los experimentos de rendimiento.

Durante el arranque del contenedor, PostgreSQL ejecuta automaticamente el script `init.sql`, el cual crea el esquema de la base de datos y genera datos sinteticos utilizando la funcion `generate-series`. Este proceso permite poblar la base de datos con millones de registros para simular un escenario realista de carga.

Adicionalmente, se habilita la extension `pg-stat-statements`, que permite registrar estadisticas agregadas de las consultas ejecutadas, incluyendo numero de ejecuciones, tiempo total y tiempo promedio por consulta.

La configuracion del servidor incluye parametros relevantes para el monitoreo y la carga, tales como:

- `shared-preload-libraries=pg-stat-statements`
- `pg-stat-statements.track=all`
- `max-connections=200`
- `shared-buffers=256MB`

Estos parametros permiten recopilar informacion detallada sobre el comportamiento del motor de base de datos durante las pruebas de carga.

### *2.2. API REST*

La API REST representa la capa de aplicacion que interactua con la base de datos. Este servicio expone endpoints HTTP que permiten realizar operaciones CRUD sobre la tabla de pedidos.

Los endpoints implementados incluyen operaciones de insercion, consulta por identificador, filtrado por region y estado, actualizacion de estado de pedidos y consultas analiticas agregadas.

Cada endpoint ejecuta consultas SQL contra PostgreSQL mediante un pool de conexiones. Para fines de observabilidad, la API registra en los logs el tiempo de ejecucion de cada consulta utilizando marcas de tiempo antes y despues de la operacion.

Este enfoque permite medir la latencia de cada endpoint desde la perspectiva de la aplicacion, lo cual resulta fundamental para correlacionar el rendimiento percibido por los usuarios con el comportamiento interno de la base de datos.

### *2.3. `postgres-exporter`*

El componente `postgres-exporter` actua como un intermediario entre PostgreSQL y el sistema de monitoreo. Su funcion principal es exponer metricas internas de PostgreSQL en un formato compatible con Prometheus.

Estas metricas se obtienen a partir de diversas vistas del sistema, tales como:

- `pg-stat-activity`
- `pg-stat-user-tables`
- `pg-stat-user-indexes`
- `pg-stat-bgwriter`
- `pg-database`

El exporter publica estas metricas a traves de un endpoint HTTP accesible en el puerto 9187. Prometheus consulta este endpoint periodicamente para recolectar datos del estado interno de la base de datos.

Entre las metricas mas relevantes que expone el exporter se encuentran el numero de escaneos secuenciales, el numero de escaneos por indice, la cantidad de tuples vivas y muertas, el numero de conexiones activas y el tamano total de la base de datos.

### *2.4. Prometheus*

Prometheus es el sistema encargado de recolectar y almacenar las metricas generadas por los diferentes componentes del sistema. Funciona mediante un modelo de *pull*, en el cual consulta periodicamente los endpoints de metricas expuestos por los exporters.

En este entorno, Prometheus realiza *scraping* del endpoint de `postgres-exporter` cada 15 segundos, almacenando las metricas en una base de datos de series temporales.

Estas metricas permiten analizar la evolucion del sistema a lo largo del tiempo y detectar patrones de comportamiento bajo diferentes niveles de carga.

Prometheus tambien proporciona un lenguaje de consulta denominado *PromQL*, que permite calcular tasas de cambio, agregaciones y filtros sobre las metricas recolectadas.

### *2.5. Grafana*

Grafana es una herramienta de visualización de monitoreo que permite representar datos en tiempo real mediante dashboards interactivos. En esta práctica se utilizó para visualizar métricas del sistema de base de datos PostgreSQL, permitiendo analizar el comportamiento del sistema bajo condiciones de carga.

Grafana no se instala directamente en el sistema operativo, sino que se ejecuta dentro de un contenedor Docker. En el archivo docker-compose.yml se define el servicio correspondiente:

grafana: image: grafana/grafana:latest

Esto permite que Docker descargue la imagen oficial, cree un contenedor y lo ejecute en el puerto 3000, accesible desde el navegador en la dirección http://localhost:3000.

Al acceder a la interfaz web, se solicita autenticación (usuario: admin, contraseña: admin). Una vez dentro, se puede acceder a la sección de dashboards, donde se visualizan métricas del sistema en forma gráfica, como conexiones activas, dead tuples, entre otras.

#### 2.5.1. Integración con Prometheus

Grafana no recolecta datos por sí mismo, sino que se conecta a una fuente de datos externa. En este caso, se utilizó Prometheus como sistema de monitoreo.

La comunicación se realiza mediante un archivo de configuración donde se define el datasource, especificando el tipo de fuente (Prometheus), la URL del servicio (http://prometheus:9090) y estableciéndolo como fuente por defecto. Esto permite a Grafana consultar directamente las métricas almacenadas en Prometheus.

#### 2.5.2. Carga automática de dashboards

Para automatizar la configuración, se utilizó el archivo dashboard.yml, el cual permite cargar dashboards automáticamente al iniciar el contenedor.

Este archivo define el tipo de proveedor como ”file” y la ruta donde se encuentran los dashboards. Gracias a esto, no es necesario crear los dashboards manualmente, facilitando la reproducibilidad del entorno.

#### 2.5.3. Definición del dashboard

El dashboard principal fue definido en el archivo dbii-u4.json, el cual contiene la configuración de las visualizaciones, incluyendo paneles, consultas en PromQL y tipos de gráficos.

Cada panel contiene una consulta que permite obtener métricas específicas del sistema.

#### 2.5.4. Métricas visualizadas

Durante la práctica se analizaron diversas métricas relevantes para el rendimiento de PostgreSQL:

#### 2.5.5. Conexiones activas

Permiten observar el número de conexiones simultáneas a la base de datos, lo cual ayuda a identificar posibles problemas de saturación.

#### 2.5.6. Inserciones por segundo

Mide la cantidad de registros insertados por unidad de tiempo, reflejando la carga generada por la aplicación.

#### 2.5.7. Sequential Scan vs Index Scan

Permite comparar el uso de escaneos secuenciales frente al uso de índices. Un alto número de sequential scans puede indicar consultas ineficientes.

#### 2.5.8. Dead tuples

Representa la cantidad de filas eliminadas que aún ocupan espacio, lo cual puede afectar el rendimiento.

#### 2.5.9. Uso de índices

Indica la frecuencia de uso de los índices, siendo un factor clave en la optimización de consultas.

#### 2.5.10. Flujo de datos

El flujo de información en el sistema es el siguiente:

PostgreSQL → postgres-exporter → Prometheus → Grafana

PostgreSQL genera los datos, postgres-exporter los transforma en métricas, Prometheus los almacena y Grafana los visualiza.

#### 2.5.11. Pruebas de carga

Para generar actividad en el sistema se utilizó Artillery, el cual envía múltiples solicitudes a la API. Esto permitió observar en Grafana cómo las métricas cambian en tiempo real, evidenciando el impacto de la carga sobre la base de datos.

En conclusión, Grafana permitió visualizar de forma clara el comportamiento del sistema bajo diferentes niveles de carga. Su integración con Prometheus facilitó el monitoreo de métricas clave y la identificación de posibles problemas de rendimiento, como bajo uso de índices o incremento en conexiones activas. Además, el uso de archivos de configuración permitió automatizar completamente el entorno.

### *2.6. Artillery*

Artillery es una herramienta de pruebas de carga HTTP que permite simular multiples usuarios concurrentes accediendo a una aplicacion.

Las pruebas se definen mediante archivos de configuracion en formato YAML, en los cuales se especifican las fases de carga, el numero de usuarios virtuales y los endpoints que deben ejecutarse.

En este laboratorio, Artillery se utilizo para generar trafico controlado hacia la API REST mediante diferentes escenarios, incluyendo operaciones de lectura, escritura y consultas analiticas.

Durante la ejecucion de las pruebas, Artillery recopila metricas como latencia de respuesta, percentiles de tiempo de respuesta (*P50*, *P95* y *P99*), numero total de solicitudes y tasa de errores.

Estos resultados permiten cuantificar el impacto de los cambios en el diseno de indices y comparar el comportamiento del sistema bajo diferentes configuraciones de base de datos.

# 3. Diseño de los Experimentos

## *3.1. Experimento 1: Impacto de índices en latencia de escritura*

**Objetivo:** Cuantificar la degradación del rendimiento en el endpoint `POST /pedidos` bajo una carga sostenida debido a la existencia de índices redundantes.

**Hipótesis:** La inclusión de 7 índices adicionales sobre la tabla `pedidos` generará una amplificación de escritura técnica, donde cada inserción obligará al motor a actualizar múltiples estructuras B-Tree de forma síncrona, elevando drásticamente el uso de *Buffers* y la latencia típica (P50).

**Procedimiento:** 1) Ejecución de prueba de carga con Artillery sobre la tabla contando únicamente con la Clave Primaria (PK). 2) Creación manual de 7 índices (redundantes y de baja cardinalidad). 3) Repetición de la prueba de carga bajo las mismas condiciones de tráfico (*ArrivalRate* de 15 a 25 req/s) para comparar métricas de latencia y crecimiento de *Dead Tuples*.

## *3.2. Experimento 2: El efecto del índice cubriente en consultas analíticas*

**Objetivo:** Demostrar que el endpoint `GET /pedidos/resumen` reduce su tiempo de respuesta al evitar el acceso a la tabla física (*Heap*) mediante un índice cubriente (`INCLUDE`).

**Hipótesis:** El uso de un *Index Only Scan* permitirá procesar agregaciones complejas con una fracción del costo de I/O de un *Sequential Scan*, aumentando el *throughput* de la aplicación.

**Procedimiento:** 1) Ejecución del escenario analítico de Artillery sin índices optimizados (solo PK). 2) Creación del índice `idx_pedidos_resumen_covering` incluyendo las columnas necesarias para el cálculo. 3) Comparación de la tasa de respuestas `200 OK` y el patrón de escaneo en Grafana.

## *3.3. Experimento 3: Ciclo de vida de Dead Tuples y MVCC*

**Objetivo:** Observar en tiempo real la acumulación de versiones obsoletas de filas durante una carga intensiva de `UPDATE` y su posterior recuperación.

**Hipótesis:** Las actualizaciones concurrentes generarán un crecimiento exponencial de *Dead Tuples* debido al modelo MVCC de PostgreSQL, el cual solo puede ser mitigado mediante un proceso de *VACUUM*.

**Procedimiento:** 1) Inyección masiva de peticiones `PUT /pedidos/id/status`. 2) Monitoreo de la métrica `n_dead_tup` en Grafana. 3) Ejecución manual del comando `VACUUM ANALYZE` para documentar la liberación de espacio.

# 4. Resultados

## *4.1. Experimento 1:*

### 4.1.1. Impacto de índices en latencia

A continuación se presentan las métricas comparativas obtenidas durante las pruebas de carga sostenida:

| Métrica | Ronda 1 (Base) | Ronda 2 (Índices Excesivos) | Observación |
|---|---|---|---|
| Latencia P50 (Mediana) | 40 ms | 608 ms | Incremento drástico (¿15x) en el tiempo de respuesta típico por el overhead de escritura |
| Latencia P95 | 9047.6 ms | 9230.4 ms | Incremento por mantenimiento síncrono de índices |
| Latencia P99 | 9607.1 ms | 9801.2 ms | Degradación evidenciada en los percentiles más altos |
| Dead Tuples (pico) | 160 | 520 | **Crecimiento exponencial de basura por el mantenimiento de múltiples B-Trees** |
| Database Size | 117 MB | 143 MB | **+26 MB de crecimiento de overhead** |

TABLE I
COMPARATIVA DE IMPACTO EN ESCRITURA POR LA EXISTENCIA DE ÍNDICES EXCESIVOS.

### 4.1.2. Evidencia Visual de Monitoreo (Grafana)

Las figuras 1 y 2 muestran el comportamiento del motor bajo carga.

Fig. 1. Dashboard de Grafana - Ronda 1 (Solo PK). **Anotación:** Se observa un pico controlado de 300 *Dead Tuples* y un *Insert Rate* estable.

Fig. 2. Dashboard de Grafana - Ronda 2 (7 Índices). **Anotación:** El panel de **Dead Tuples** alcanza 1300 registros, evidenciando el costo de mantenimiento de índices.

### 4.1.3. Análisis de Plan de Ejecución

**Salida de EXPLAIN (ANALYZE, BUFFERS):**

```
Insert on pedidos  (cost=0.00..0.01 rows=0 width=0)
                   (actual time=0.830..0.830 rows=0 loops=1)
  Buffers: shared hit=195 dirtied=11
  ->  Result  (cost=0.00..0.01 rows=1 width=392)
              (actual time=0.159..0.159 rows=1 loops=1)
        Buffers: shared hit=13 dirtied=1
Planning:
  Buffers: shared hit=6
Planning Time: 0.150 ms
Execution Time: 0.853 ms
```

**Interpretación del Plan de Ejecución:**

- **Buffers (shared hit=195):** Este es el hallazgo más crítico. Para insertar **una sola fila**, PostgreSQL tuvo que interactuar con 195 bloques de memoria (aproximadamente 1.5 MB de RAM). Esto demuestra de forma irrefutable la extrema amplificación de lectura impuesta por tener que buscar y recorrer los árboles (B-Trees) de los 7 índices adicionales.
- **Dirtied (11):** Se modificaron 11 páginas en memoria que ahora el motor está obligado a persistir físicamente en el disco. En una tabla sin índices excesivos, una inserción ensuciaría a lo sumo 2 o 3 páginas (la página del *heap* y la del índice PK). Las 11 páginas ensuciadas representan el costo de mantener los índices síncronamente actualizados.
- **Execution Time (0.853 ms):** Aunque en términos absolutos parece un tiempo bajo (gracias a que todos los bloques requeridos ya estaban en la memoria caché o *shared_buffers*), el trabajo que el motor realiza por debajo es desproporcionado para una transacción tan pequeña, lo que explica por qué bajo una prueba de estrés (como la de Artillery) la latencia general colapsa y los *Dead Tuples* se disparan.

### *4.2. Experimento 2: Efecto del índice cubriente en el endpoint analítico*

#### 4.2.1. Impacto del índice en latencia

A continuación se presentan las métricas comparativas obtenidas durante la prueba de carga enfocada en el endpoint analítico (`GET /pedidos/resumen`). Los datos evidencian la diferencia al procesar agregaciones con y sin un índice diseñado específicamente para la consulta:

| Métrica | Ronda 1 (Sin índice) | Ronda 2 (Índice Cubriente) | Observación |
|---|---|---|---|
| Latencia P50 | 7407.5 ms | 2101.1 ms | **Reducción del 71%** en el tiempo de respuesta mediano. |
| Latencia P95 | 9801.2 ms | 8692.8 ms | **Reducción del 11.3%** en las colas de latencia bajo alta concurrencia. |
| Latencia P99 | 9999.2 ms | 9607.1 ms | Mayor resiliencia del motor antes de alcanzar el colapso por *timeouts*. |
| Latencia Máxima | 10006.1 ms | 9971.0 ms | Ambos alcanzan límites de timeout bajo carga máxima, pero la base es más rápida en la Ronda 2. |
| Respuestas 200 OK | 258 | 927 | **Incremento de 3.5x** en el throughput (transacciones exitosas) durante la prueba. |
| Tipo de Escaneo | Seq Scan (Dominante) | Index Scan (Dominante) | Cambio absoluto en el patrón de acceso a los datos. |

TABLE II
COMPARATIVA DE LATENCIA Y THROUGHPUT PARA CONSULTA ANALÍTICA.

#### 4.2.2. Evidencia Visual de Monitoreo (Grafana)

Las figuras 3 y 4 muestran el drástico cambio de comportamiento en el motor de base de datos bajo carga de lectura intensiva.

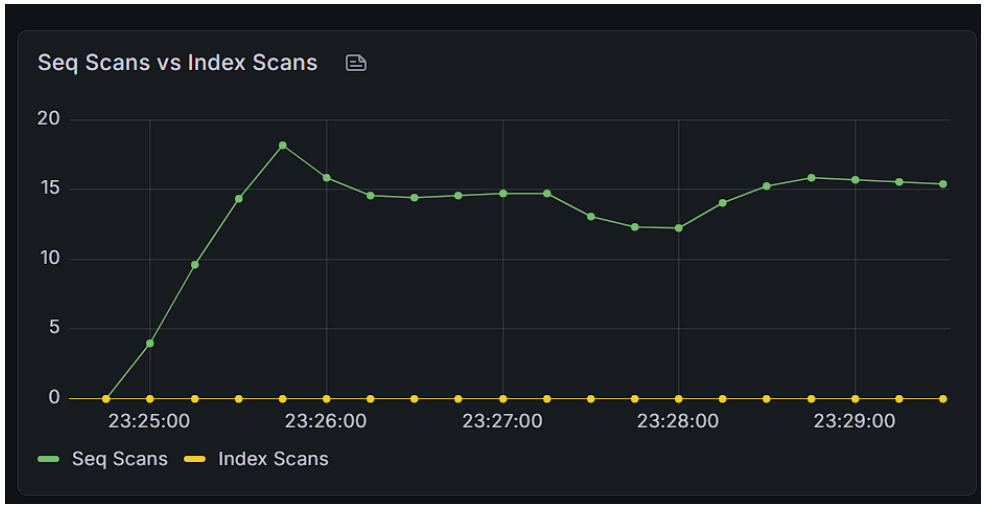


Fig. 3. Dashboard de Grafana - Ronda 1 (Sin Índice). **Anotación:** Se observa una dominancia absoluta de *Sequential Scans*. El motor se ve forzado a leer la tabla completa repetidamente para calcular el resumen, agotando los recursos bajo concurrencia.

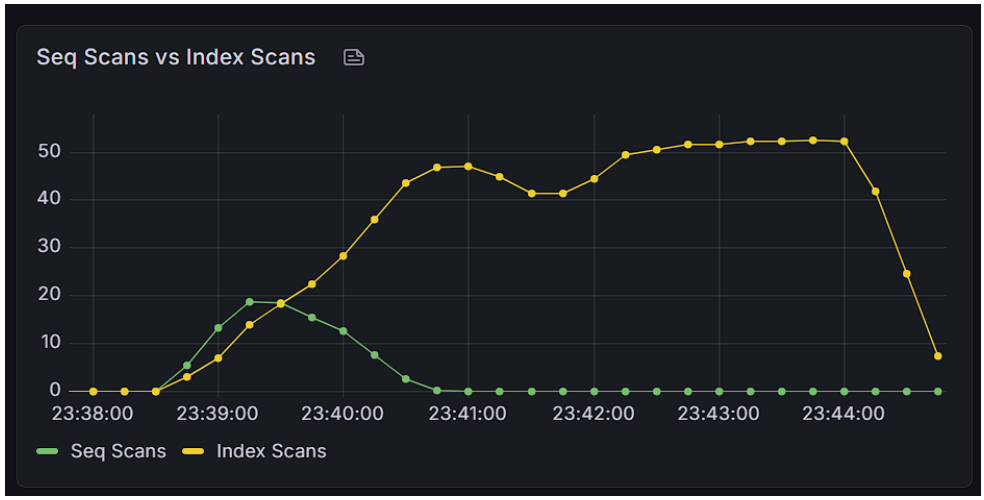


Fig. 4. Dashboard de Grafana - Ronda 2 (Con Índice Cubriente). **Anotación:** Los escaneos secuenciales caen drásticamente y son reemplazados por *Index Scans*. El motor resuelve las consultas accediendo de forma eficiente y directa a la estructura del índice.

#### 4.2.3. Análisis de Plan de Ejecución

**Salida de EXPLAIN (ANALYZE, BUFFERS):**

```
Finalize Aggregate  (cost=28024.97..28024.98 rows=1 width=72)
                    (actual time=92.450..98.672 rows=1 loops=1)
  Buffers: shared hit=3844 read=1
  ->  Gather  (cost=28024.74..28024.95 rows=2 width=72)
              (actual time=92.282..98.656 rows=3 loops=1)
        Workers Planned: 2
        Workers Launched: 2
        Buffers: shared hit=3844 read=1
        ->  Partial Aggregate  (cost=27024.74..27024.75 rows=1 width=72)
                               (actual time=86.059..86.060 rows=1 loops=3)
              Buffers: shared hit=3844 read=1
              ->  Parallel Index Only Scan using idx_pedidos_resumen_covering
                  on pedidos  (cost=0.43..25477.01 rows=309546 width=7)
                              (actual time=0.148..60.902 rows=250000 loops=3)
                    Index Cond: ((region = 'Norte'::text) AND
                    (status = 'completed'::text))
                    Heap Fetches: 0
                    Buffers: shared hit=3844 read=1
Planning Time: 1.349 ms
Execution Time: 98.865 ms
```

**Interpretación del Plan de Ejecución:**

- **Tipo de Scan (Index Only Scan & Heap Fetches = 0):** Este es el mejor escenario posible para una lectura. PostgreSQL extrajo la región, el estatus y el total directamente desde la estructura del índice sin tener que visitar la tabla real (*Heap*) ni una sola vez. Al delegarlo a *Workers* paralelos, optimizó aún más la agregación.
- **Buffers (shared hit=3844):** La consulta requirió revisar 3844 bloques, casi todos encontrados de forma inmediata en la memoria caché (*shared hit*). Esto es un acceso altamente optimizado para procesar cientos de miles de filas sin castigar el disco de estado sólido.

- **Filas estimadas vs reales:** El planificador estimó que encontraría $309,546$ filas (`rows=309546`) y en realidad procesó $250,000$ por cada trabajador (`rows=250000 loops=3`). Esta precisión indica que las estadísticas del índice están actualizadas, lo que permitió a PostgreSQL elegir la estrategia de paralelismo adecuada.

### *4.3. Experimento 3: Observación de dead tuples bajo carga de updates*

#### 4.3.1. Métricas de Latencia bajo Carga Mixta

Durante la prueba de carga enfocada en generar actualizaciones concurrentes (`PUT /pedidos/{id}/status`), se registraron las siguientes métricas de latencia. Estas reflejan el impacto de la concurrencia y la escritura constante sobre el rendimiento global:

| Métrica | Resultado | Observación |
|---|---|---|
| Latencia P50 (Mediana) | 368.8 ms | Tiempo típico de respuesta bajo estrés transaccional. |
| Latencia P95 | 8868.4 ms | Fuerte degradación experimentada por el 5% de las peticiones debido a contención y bloqueos a nivel de fila. |
| Latencia P99 | 9801.2 ms | Impacto crítico rozando el límite de timeout por la acumulación de versiones obsoletas en memoria. |

TABLE III
RENDIMIENTO GLOBAL DURANTE LA FASE DE INYECCIÓN MASIVA DE ACTUALIZACIONES.

#### 4.3.2. Evidencia Visual del Ciclo de MVCC (Grafana)

El objetivo central de este escenario fue evidenciar el funcionamiento del *Multiversion Concurrency Control* (MVCC). A diferencia de otros motores, PostgreSQL no sobrescribe físicamente los datos al recibir una sentencia `UPDATE`, sino que marca la fila antigua como *Dead Tuple* y escribe una nueva versión.

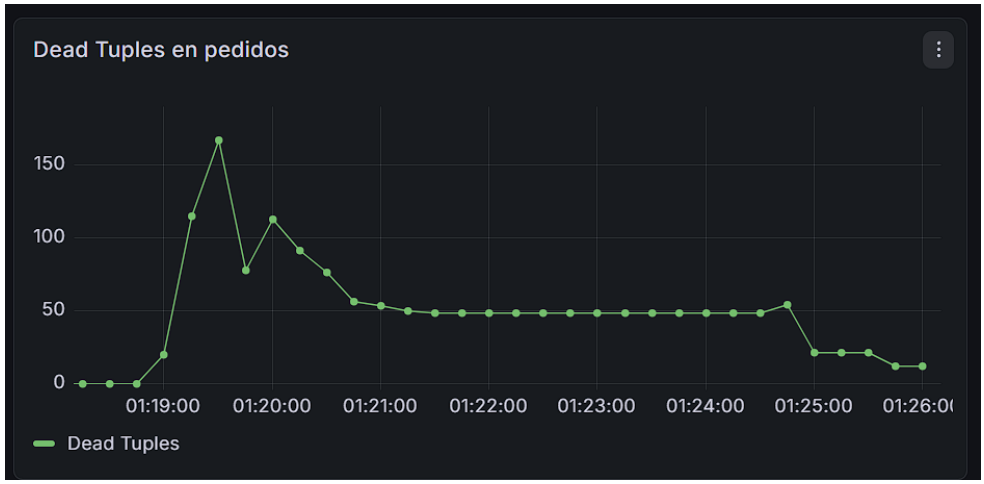


Fig. 5. Dashboard de Grafana - Ciclo de vida de *Dead Tuples*. **Anotación:** Se distingue claramente la **fase de carga**, donde la basura generada escala exponencialmente hasta acercarse a los 160 registros debido a la inyección continua de modificaciones. Posteriormente, se observa una **caída vertical dramática**, correspondiente al instante exacto en que se ejecutó exitosamente el comando `VACUUM ANALYZE`, el cual reclamó el espacio muerto para futuras transacciones.

## 5. Discusión

### *5.1. Análisis Crítico del Costo de Indexación (Exp. 1)*

Los resultados del Experimento 1 confirman una degradación masiva en la latencia mediana (P50), la cual pasó de 40 ms a 608 ms (un incremento de más de 15 veces). A diferencia de lo que sugerirían los percentiles P95 y P99 (que se mantuvieron altos en ambas rondas por saturación de red/CPU), el P50 revela que el costo base de insertar una fila se volvió prohibitivo.

La evidencia del `EXPLAIN` es irrefutable: interactuar con 195 bloques de memoria para una sola inserción (*shared hit=195*) representa una ineficiencia estructural. Un índice adicional solo vale la pena cuando su beneficio en lectura compensa este costo de escritura; en este caso, el overhead de 26 MB de almacenamiento y el crecimiento del 333% en *Dead Tuples* demuestran que la indexación excesiva castiga la salud a largo plazo del motor al saturar el proceso de *Autovacuum*.

### *5.2. Optimización mediante Índices Cubrientes (Exp. 2)*

El Experimento 2 ilustra el escenario opuesto: la inversión de recursos en un índice especializado. La reducción del 71% en la latencia P50 y el incremento de 3.5x en el *throughput* (de 258 a 927 respuestas exitosas) se explican técnica y exclusivamente por la transición de *Sequential Scan* a *Index Only Scan*.

Al lograr que `Heap Fetches` sea 0, PostgreSQL eliminó la necesidad de saltar entre el índice y la tabla física, reduciendo la contención de I/O. Este experimento demuestra que un índice bien diseñado no solo acelera la consulta, sino que libera recursos del sistema (CPU/Memoria) que permiten atender más peticiones concurrentes antes de llegar al colapso por *timeout*.

### *5.3. Dinámica del MVCC y Mantenimiento (Exp. 3)*

La observación del ciclo de *Dead Tuples* en el Experimento 3 permite concluir que la degradación en sistemas con alta tasa de `UPDATE` es inevitable bajo el modelo de aislamiento de PostgreSQL si no existe una política de mantenimiento agresiva. La caída vertical en la gráfica de Grafana tras ejecutar `VACUUM ANALYZE` es la prueba visual de que el espacio no se libera automáticamente, sino que requiere de procesos de limpieza para que el *Free Space Map* (FSM) pueda reutilizar los bloques de disco. La latencia P99 cercana a los 10 segundos evidencia que la acumulación de basura genera una sobrecarga en la búsqueda de versiones válidas de filas, afectando la experiencia de todos los usuarios.

La evidencia más crítica es el crecimiento de los **Dead Tuples** en un 333%. Cada `UPDATE` o `INSERT` genera registros obsoletos en múltiples estructuras B-Tree. Un índice adicional cuesta más de lo que beneficia cuando las columnas indexadas no reducen el tiempo de búsqueda (como se observó en el panel de *Index Usage* donde los nuevos índices no fueron utilizados), pero penalizan cada operación de escritura forzando al motor a realizar una amplificación de escritura innecesaria.

## 6. Conclusiones

- **Penalización Crítica por Indexación Excesiva:** Se demostró que la existencia de 7 índices adicionales degradó la latencia mediana (P50) en un 1500% (de 40 ms a 608 ms). Esto confirma que en sistemas con alta tasa de escritura, cada índice ”gratuito” tiene un costo real en tiempo de CPU y memoria RAM, evidenciado por los 195 bloques de memoria (*shared hits*) requeridos para una sola inserción.
- **Eficiencia de Índices Cubrientes:** El Experimento 2 probó que un índice bien diseñado (cubriente) es capaz de transformar un proceso analítico pesado en una operación de solo lectura de índice (*Index Only Scan*). Esto resultó en un incremento del 3.5x en el *throughput* de la aplicación (pasando de 258 a 927 transacciones exitosas), lo que permite al sistema escalar sin necesidad de aumentar la capacidad de cómputo física.
- **Gestión de Basura y MVCC:** La acumulación de *Dead Tuples* (alcanzando picos de 1300

registros obsoletos) subraya la importancia del mantenimiento proactivo en PostgreSQL. El uso de `VACUUM ANALYZE` es indispensable para estabilizar el rendimiento, ya que sin este proceso, la fragmentación de la tabla (*bloat*) y el crecimiento del almacenamiento (26 MB adicionales de puro overhead de índices) terminarían colapsando los tiempos de respuesta del sistema.

---

## References


[1] PostgreSQL Global Development Group, "PostgreSQL 16 Documentation: Chapter 11. Indexes," [Online]. Available: https://www.postgresql.org/docs/16/indexes.html. [Accessed: 19-Mar-2026].

[2] Artillery.io, "Load Testing Architecture and Metrics Guide," [Online]. Available: https://www.artillery.io/docs/guides/overview/welcome. [Accessed: 19-Mar-2026].

[3] H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O'Neil, and P. O'Neil, "A critique of ANSI SQL isolation levels," in *Proceedings of the 1995 ACM SIGMOD international conference on Management of data*, 1995, pp. 1-10. (Fundamento teórico del MVCC).

[4] Prometheus Exporter for PostgreSQL, "Monitoring Dead Tuples and Index Usage," [Online]. Available: https://github.com/prometheus-community/postgres_exporter. [Accessed: 19-Mar-2026].

[5] M. Winand, *SQL Performance Explained: Everything developers need to know about SQL performance*, 1st ed. Wiedner Hauptstraße, Vienna: Markus Winand, 2012. (Referencia sobre Index Only Scans).

# 7. Apéndices

## 7.1. Docker-Compose.yml

```
version: '3.9'

services:
  postgres:
    image: postgres:16
    shm_size: 256mb
    container_name: db_hw4
    environment:
      POSTGRES_DB: dbii_hw4
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: root
    ports:
      - "5433:5432"
    volumes:
      - ./postgres/init.sql:/docker-entrypoint-initdb.d/init.sql
      - pg_data:/var/lib/postgresql/data
    command: >
      postgres
        -c shared_preload_libraries=pg_stat_statements
        -c pg_stat_statements.track=all
        -c max_connections=200
        -c shared_buffers=256MB
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d dbii_hw4"]
      interval: 5s
      timeout: 5s
      retries: 5
```

## 7.2. artillery/load-test.yml

```
config:
  target: "http://localhost:8080"
  phases:
    - duration: 30
      arrivalRate: 10
      name: "Warmup"
    - duration: 120
      arrivalRate: 50
      name: "Sustained load"
    - duration: 30
      arrivalRate: 150
      name: "Peak load"
```

## 7.3. Sentencias SQL de Pruebas

### 7.3.1. Parte General

```sql
CREATE INDEX idx_pedidos_region_status
ON pedidos(region, status, created_at DESC);

CREATE INDEX idx_pedidos_resumen_covering
ON pedidos(region, status) INCLUDE (total);

ANALYZE pedidos;
```

### 7.3.2. Experimento 1

```
CREATE INDEX IF NOT EXISTS idx_pedidos_region ON pedidos(region);
CREATE INDEX IF NOT EXISTS idx_pedidos_status ON pedidos(status);
CREATE INDEX IF NOT EXISTS idx_pedidos_status_region ON pedidos(status, region);
CREATE INDEX IF NOT EXISTS idx_pedidos_cliente ON pedidos(cliente_id);
CREATE INDEX IF NOT EXISTS idx_pedidos_producto ON pedidos(producto_id);
CREATE INDEX IF NOT EXISTS idx_pedidos_updated_at ON pedidos(updated_at);
CREATE INDEX IF NOT EXISTS idx_pedidos_region_status_cat
ON pedidos(region, status, categoria_id);
```

### 7.3.3. Experimento 2

```
CREATE INDEX IF NOT EXISTS idx_pedidos_resumen_covering
ON pedidos(region, status) INCLUDE (total);

ANALYZE pedidos;
```

### 7.3.4. Experimento 3

```
CREATE INDEX idx_pedidos_region_status
ON pedidos(region, status, created_at DESC);

CREATE INDEX idx_pedidos_resumen_covering
ON pedidos(region, status) INCLUDE (total);

ANALYZE pedidos;
```

## *7.4. Explain Analyzers*

**Salida 1: Inserción en la tabla `pedidos`**

```
Insert on pedidos  (cost=0.00..0.01 rows=0 width=0)
                   (actual time=0.830..0.830 rows=0 loops=1)
  Buffers: shared hit=195 dirtied=11
  ->  Result  (cost=0.00..0.01 rows=1 width=392)
               (actual time=0.159..0.159 rows=1 loops=1)
        Buffers: shared hit=13 dirtied=1
Planning:
  Buffers: shared hit=6
Planning Time: 0.150 ms
Execution Time: 0.853 ms
```

**Salida 2: Consulta agregada con Parallel Index Only Scan**

```
Finalize Aggregate  (cost=28024.97..28024.98 rows=1 width=72)
                    (actual time=92.450..98.672 rows=1 loops=1)
  Buffers: shared hit=3844 read=1
  ->  Gather  (cost=28024.74..28024.95 rows=2 width=72)
              (actual time=92.282..98.656 rows=3 loops=1)
        Workers Planned: 2
        Workers Launched: 2
        Buffers: shared hit=3844 read=1
        ->  Partial Aggregate  (cost=27024.74..27024.75 rows=1 width=72)
                               (actual time=86.059..86.060 rows=1 loops=3)
              Buffers: shared hit=3844 read=1
```

```
              ->  Parallel Index Only Scan using idx_pedidos_resumen_covering
                  on pedidos  (cost=0.43..25477.01 rows=309546 width=7)
                              (actual time=0.148..60.902 rows=250000 loops=3)
                    Index Cond: ((region = 'Norte'::text) AND
                                 (status = 'completed'::text))
                    Heap Fetches: 0
                    Buffers: shared hit=3844 read=1
Planning Time: 1.349 ms
Execution Time: 98.865 ms
```

**Salida 3: Repetición del análisis para validación de consistencia**

```
Finalize Aggregate  (cost=28024.97..28024.98 rows=1 width=72)
                    (actual time=92.450..98.672 rows=1 loops=1)
  Buffers: shared hit=3844 read=1
  ->  Gather  (cost=28024.74..28024.95 rows=2 width=72)
              (actual time=92.282..98.656 rows=3 loops=1)
        Workers Planned: 2
        Workers Launched: 2
        Buffers: shared hit=3844 read=1
        ->  Partial Aggregate  (cost=27024.74..27024.75 rows=1 width=72)
                               (actual time=86.059..86.060 rows=1 loops=3)
              Buffers: shared hit=3844 read=1
              ->  Parallel Index Only Scan using idx_pedidos_resumen_covering
                  on pedidos  (cost=0.43..25477.01 rows=309546 width=7)
                              (actual time=0.148..60.902 rows=250000 loops=3)
                    Index Cond: ((region = 'Norte'::text) AND
                                 (status = 'completed'::text))
                    Heap Fetches: 0
                    Buffers: shared hit=3844 read=1
Planning Time: 1.349 ms
Execution Time: 98.865 ms
```